

STA5073Z (Data Science for Industry) Assignment 2

Authors: Manenzhe Dakalo MNNDAK002
Troy Bisnath BSNTRO001

October 13, 2026

1. Data exploration

The file contains 25,000 grayscale images of the digits 0–4, each 28 by 28 pixels with one channel, and pixel values from 0 to 255. The metadata give `image_id`, `person_id`, `digit`, `session` and `replicate`. There are 20 writers, each with five sessions, and each session contains 50 replicates of every digit, so every writer–session–digit cell has exactly 50 images (1,250 images per writer and 5,000 per digit). There are no missing values and the image IDs match the row order of the image array. The rows are sorted by writer, then digit, then session, then replicate. A recording session is identified by writer *and* session together, because session numbers repeat across writers.

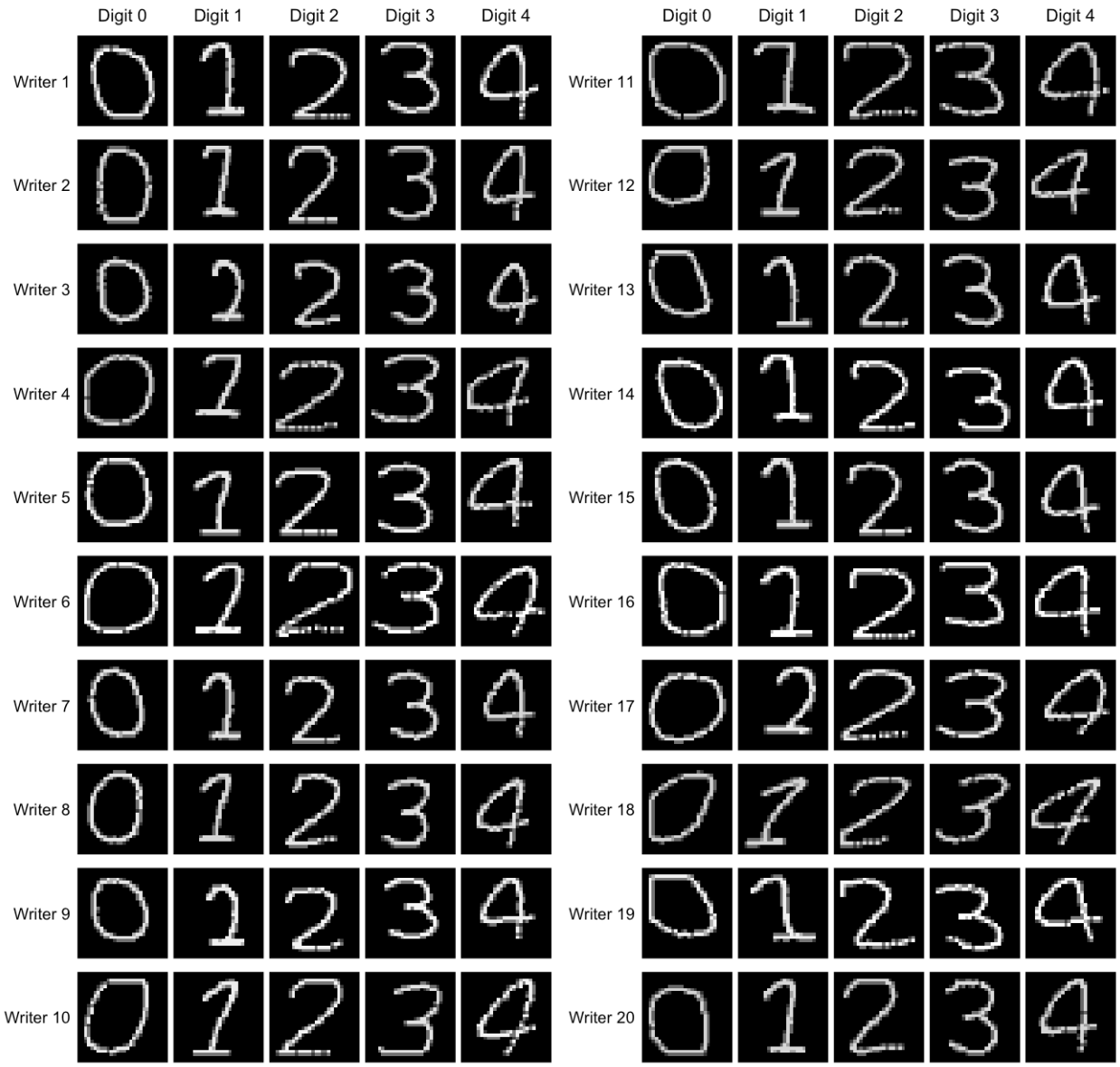


Figure 1: All 20 writers (rows) and digits 0–4 (columns), session 1, replicate 1.

Between writers. Figure 1 holds session and replicate fixed. Writers differ in size, width, slant

and in how they build each digit: writer 4 writes wide, slanted forms; writer 3 writes small, compact digits; writer 6's fours are open, while writer 7's are closed. Some writers put a bar at the bottom of the one and some do not.

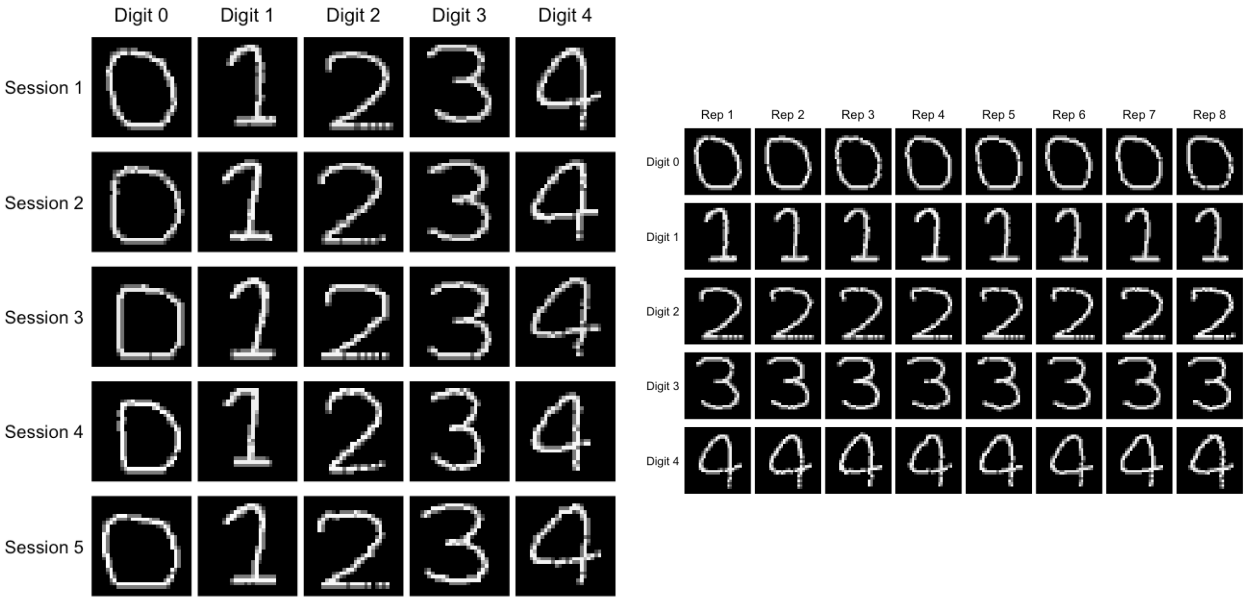


Figure 2: Writer 1 across sessions 1–5 (rows), replicate 1 (left); eight replicates within writer 1, session 1 (right).

Between sessions and within a session. Across sessions (Figure 2, left), writer 1's style is recognisable but the proportions change: the zero is rounder in some sessions and flatter in others, and the two and four change width. Within one session (Figure 2, right), replicates differ only by small shifts in outline, stroke position and brightness.

To put numbers on these sources of variation, we measured three simple features for every image: ink (mean pixel value), height and width (number of rows and columns containing a pixel brighter than 50). Table 1 compares how much these features vary between writers, between the sessions of one writer, and between replicates within a session. In each case we computed the standard deviation (SD) separately for each digit and averaged over the digits.

Table 1: Average standard deviation of simple image features at three levels.

Feature	Between writers	Between sessions (same writer)	Within a session
Ink (mean pixel value)	2.18	0.76	0.66
Height (pixels)	1.17	0.49	0.23
Width (pixels)	2.09	0.98	0.33

Writer differences are largest, as expected. More importantly, the variation between a writer's sessions is 2.1 to 3.0 times larger than the variation within a session for height and width. A session therefore shifts all of a writer's images together, so images from the same session are much

more alike than images of the same writer from different sessions. This matters for every later split.

Limitations of the simulated data. The digits are clean strokes on a black background with no paper texture, ruling, noise or neighbouring characters. Only digits 0–4 and 20 writers are included, and every cell is perfectly balanced. Shapes look like variations on a small number of templates, so real handwriting would be far more diverse. At 28 by 28 pixels, fine stroke detail (pressure, pen lifts) is lost. The session changes are simulated, so we cannot tell which real-world cause (pen, paper, mood) they represent. Results should therefore be read as results about this simulation, not about handwriting in general.

2. CNN writer identification

Prediction task. Given one digit image, predict which of the 20 writers wrote it. Only pixels are used as input; the digit and session labels are not given to the model. The response is coded 0–19 and one-hot encoded with `to_categorical()`, as in the CNN notes.

Splits. Training uses sessions 1–3, replicates 1–40 (12,000 images). Session 4 is the validation set, used to choose hyperparameters, and session 5 is the test set, used once for the chosen model (5,000 images each). We also held back replicates 41–50 from sessions 1–3 (3,000 images). These come from recordings the model has seen, but the images themselves were never used for training. We call them the *same-session check*. Section 4 explains why.

Model. The architecture follows the convolution, pooling and MLP structure in the notes. It has a 3-by-3 convolution with 32 filters and ReLU, followed by 2-by-2 max pooling, then a 3-by-3 convolution with 64 filters and ReLU, followed by 2-by-2 max pooling. The result is flattened and passed to a dense layer of 128 ReLU units, then dropout and a 20-unit softmax output. The model is compiled with categorical cross-entropy and the Adam optimiser and trained for a fixed 20 epochs with batch size 64, using session 4 as `validation_data`. The learning rate (0.0001) and dropout rate (0.5) were chosen on validation accuracy in Section 5.

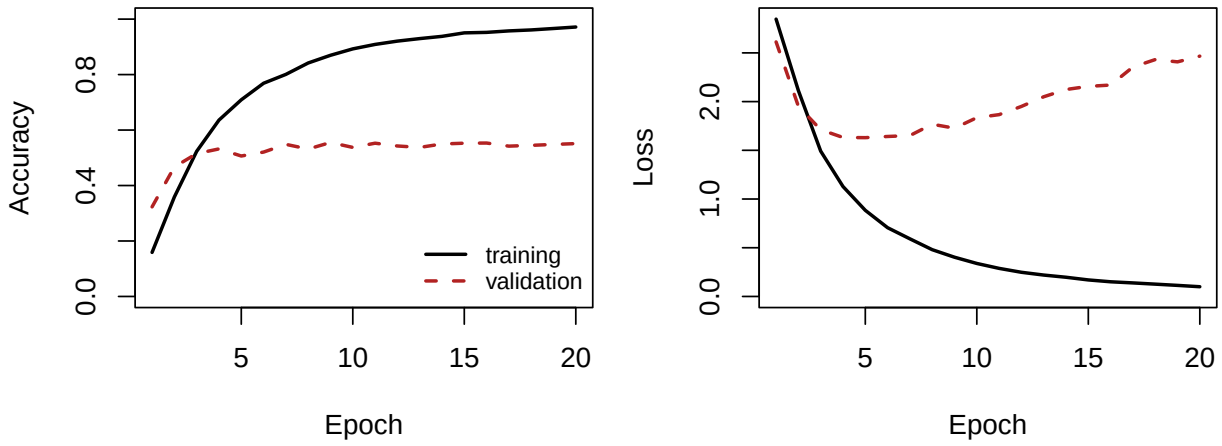


Figure 3: Training curves of the chosen CNN: accuracy (left) and cross-entropy loss (right) for the training sessions and the validation session.

Training accuracy keeps rising towards 100% (and is 100.0% when the trained model predicts the training images without dropout). Validation accuracy levels off near 55% and validation loss is lowest at epoch 5 and rises after that. This is the overfitting pattern from the notes: the model keeps fitting the training sessions but stops improving on a new one.

Results. On the test session the model identifies the correct writer for **56.1%** of images (2806 of 5000), compared with 5% for random guessing among 20 writers. Test cross-entropy is 1.86. Validation accuracy was 55.1%, close to the test accuracy, so the choice of model did not flatter the test result.

Table 2: CNN writer identification on the test session, by the digit shown.

Digit	Images	Correct	Accuracy
0	1000	600	60.0%
1	1000	519	51.9%
2	1000	486	48.6%
3	1000	665	66.5%
4	1000	536	53.6%

By digit (Table 2), the 3s identify their writer most often (66.5%) and the 2s least often (48.6%). These are all predictions from the same model, so the comparison shows how much writer information the model can extract from each digit.

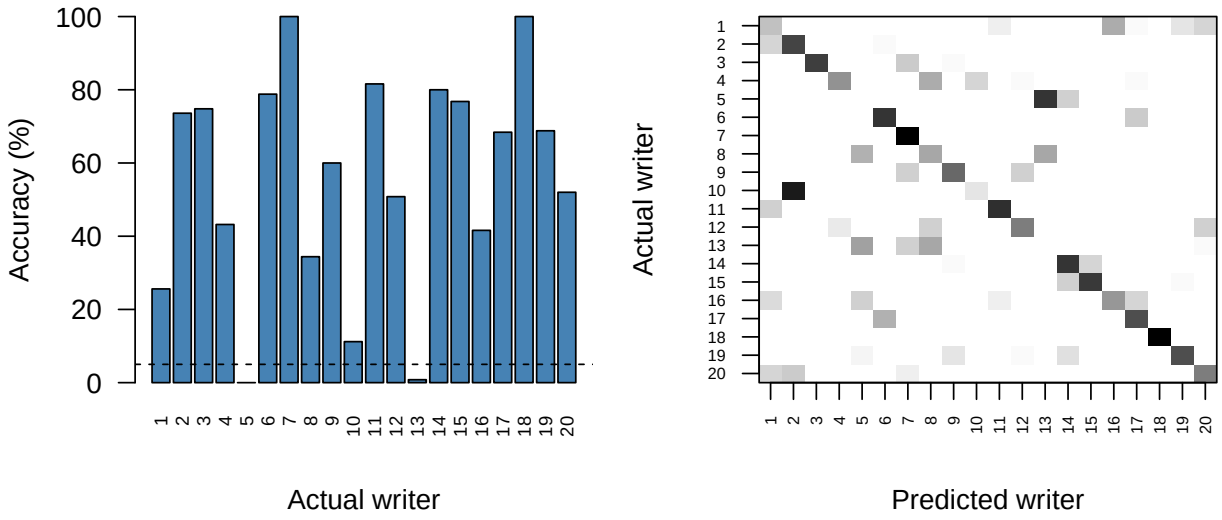


Figure 4: Left: test accuracy for each writer (dashed line: 5% chance). Right: confusion table of actual against predicted writer for the test session, as row percentages.

Per-writer accuracy varies widely (Figure 4). Writers 7 and 18 are always identified, while writer 5 is correctly identified in 0% of their test images and writer 13 in 1%. The confusion table shows where the errors go. The largest are writer 10 labelled as writer 2 (222 of 250 images), writer 5 as writer 13 (198) and writer 13 as writer 5 (95). These errors are concentrated, not random. Section 6 shows they are caused by session changes.

What the measures mean in practice. Accuracy answers this question: if one of these 20 people writes a single digit on a new occasion, how often does the system name the right person? The answer is 56% of the time. The per-writer accuracies and the confusion table matter in practice too, because a system that blames writer 2 for writer 10's writing makes the same mistake every time, rather than occasionally. The model can only choose among the 20 writers it was trained on; it cannot say "none of these".

3. Siamese network for writer matching

The Siamese network follows the metric-learning notes. A shared CNN encoder maps an image to a 32-number embedding: a 3-by-3 convolution (32 filters, ReLU), 2-by-2 max pooling, a 3-by-3 convolution (64 filters, ReLU), global average pooling and a dense layer with 32 outputs. The notes use 16 and 32 filters with a 16-number embedding; we doubled these because telling writers apart is harder than telling digits apart. Both images in a pair pass through the same encoder, so the two branches share weights. A lambda layer returns the Euclidean distance between the two embeddings. The model is trained with the contrastive loss

$$L = yD^2 + (1 - y)\max(0, 1 - D)^2,$$

where $y = 1$ for a pair from the same writer and D is the distance. Same-writer pairs are pulled together; different-writer pairs are pushed apart until they are at least the margin (1) apart. We used Adam (learning rate 0.001), batch size 64 and 20 epochs. Triplet loss would be an alternative: it compares an anchor with a positive and a negative image at once. We chose contrastive loss because it works directly with labelled pairs.

Constructing pairs. Writers are split *before* any pairs are made: 12 writers for training, 4 for validation (3, 7, 18 and 20) and 4 for testing (9, 14, 16 and 19). Within each group, `make_pairs()` from the notes picks a first image at random and then, with probability one half, a second image by the same writer (otherwise by a different writer in the same group); an image is never paired with itself. This gave 20,000 training pairs (50.3% same-writer), 4,000 validation pairs and 8,000 test pairs. Because partners are drawn from all of a writer's images, pairs naturally mix same and different digits and same and different sessions.

Should pairs control for digit identity? If same-writer pairs usually showed the same digit and different-writer pairs did not, the network could learn "same digit" as a shortcut for "same writer". Here the partner is chosen without regard to digit, so in the test set 20% of same-writer pairs and 20% of different-writer pairs show the same digit. Digit agreement therefore tells the network nothing about the label, and the network has to use style. Digit is still controlled at evaluation time: we report same-digit and different-digit pairs separately, and test each digit on its own (Section 6).

In the test set, 20% of same-writer pairs come from the same session, which allows the session comparison in Section 6. For the digit comparison we also built five extra test sets of 2,000 pairs each, one per digit, using only test-writer images of that digit.

Threshold and evaluation. The model outputs a distance, not a decision. As in the notes, we tried every observed validation distance as a threshold and chose the one with the highest balanced accuracy: the average of the accuracy on same-writer pairs and the accuracy on different-writer

pairs. The chosen threshold is 0.606; a pair is called “same writer” if its distance is below it. The threshold was then fixed and applied to the test pairs.

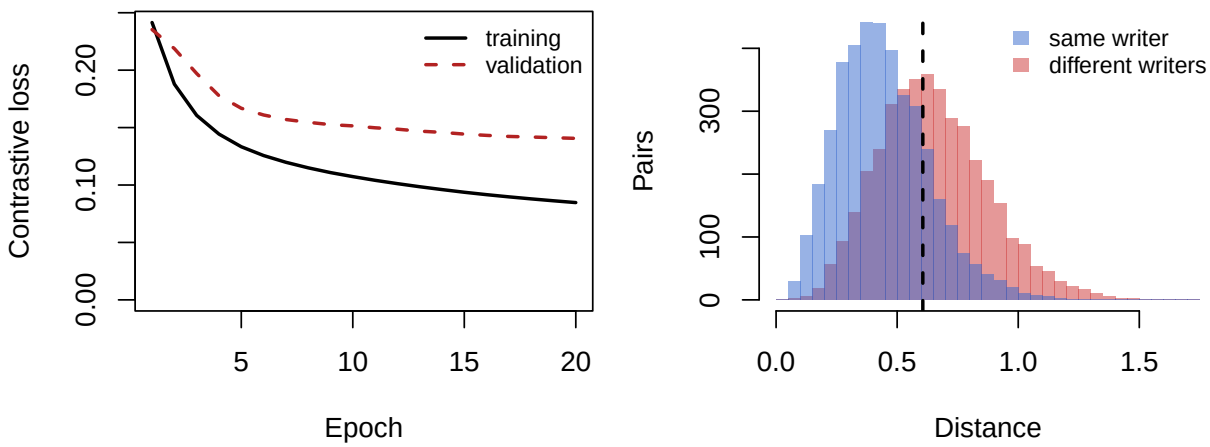


Figure 5: Left: contrastive loss by epoch. Right: test distances for same-writer and different-writer pairs, with the validation-chosen threshold.

Table 3: Test pairs (four unseen writers): actual against predicted.

	Same	Different
Same writer	3305	739
Different writers	1795	2161

On the 8,000 test pairs from unseen writers, the network accepts 81.7% of same-writer pairs and correctly rejects 54.6% of different-writer pairs, giving a balanced accuracy of **68.2%** (chance is 50%). In practice, if the system is asked “did the same person write these two digits?” about two people it has never seen, it confirms a true match 82% of the time and wrongly links two different people 45% of the time. It is therefore much better at confirming genuine matches than at excluding different writers. The two distance distributions in Figure 5 overlap heavily, which is why no threshold can avoid both kinds of error. On the validation writers the same threshold gave 81.4% balanced accuracy, 13 points higher than on the test writers. Section 4 explains this gap.

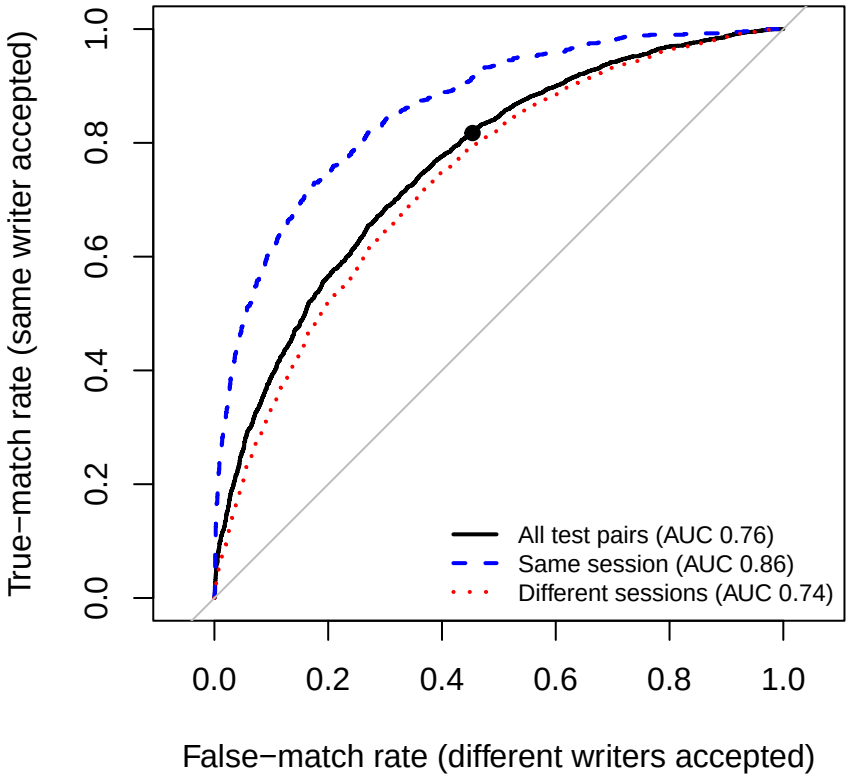


Figure 6: ROC curves for the test pairs: all pairs, and same-writer pairs from one session or from two different sessions (each against all different-writer pairs). The dot marks the validation-chosen threshold, the grey diagonal is chance.

Threshold-free evaluation. Balanced accuracy is a threshold-dependent metric. The ROC curve (Figure 6) displays all feasible thresholds at once: each point indicates the proportion of true matches accepted against the proportion of different-writer pairs falsely accepted. The area under the curve (AUC) is the probability that a randomly chosen different-writer pair is further apart than a randomly chosen same-writer pair, so 0.5 is chance and 1 is perfect separation. The AUC is **0.76** on the test writers and 0.89 on the validation writers. The equal error rate is 30.9%: this is the error rate at the threshold where false matches and missed matches are equally common, and it is the traditional single-number summary for verification systems such as signature checking. The curves also separate by session: the AUC is 0.86 when a same-writer pair comes from one session and 0.74 when it comes from two. The session effect addressed in Section 6 is therefore not a side-effect of where the threshold was placed.

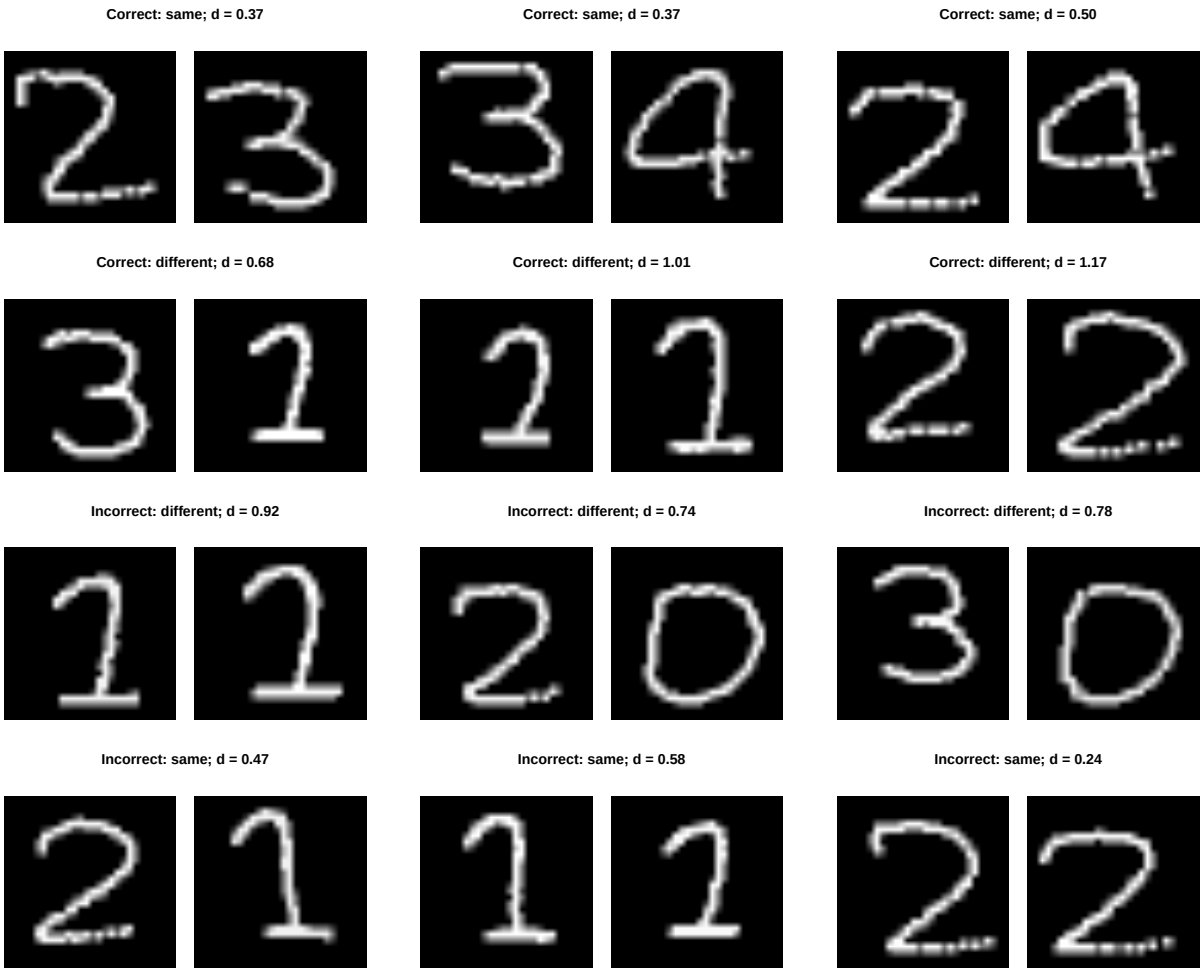


Figure 7: Example test pairs in each outcome group, with their distances.

4. Training, validation and test strategy

Table 4: Data splits.

Model	Set	Contents	Purpose
CNN	Training	Sessions 1-3, replicates 1-40, all writers	Fit weights
	Same-session check	Sessions 1-3, replicates 41-50, all writers	Measure performance on seen recordings
	Validation	Session 4, all writers	Choose learning rate and dropout
	Test	Session 5, all writers	Final estimate, used once
Siamese	Training	Pairs from 12 writers	Fit weights
	Validation	Pairs from 4 other writers	Choose the distance threshold
	Test	Pairs from 4 further writers	Final estimate, used once

The deck-3 notes say to fit on training data, compare settings on validation data and keep the test data separate until the final model is chosen. We followed that rule. The bigger decision is *what* to hold out, because images from the same writer and session are strongly related (Table 1).

CNN: new sessions of known writers. A 20-class classifier can only name the writers it was trained on, so every writer must appear in training. What can be held out is the *occasion*. Holding out whole sessions measures the realistic task of recognising a known person's writing on a new day. The same-session check shows what happens otherwise. On held-back replicates from the training sessions, the CNN scores **100.0%**, against 56.1% on the new session. A random split of images, or Keras' `validation_split`, would therefore have reported about 100% accuracy, a figure that does not survive a change of session. Because the rows are sorted by writer, `validation_split = 0.2` would be wrong here for a second reason: it takes the *last* 20% of rows, which would hold out writers 17–20 entirely. We therefore passed an explicit `validation_data` set.

Siamese: unseen writers. The Siamese network learns a distance rather than a list of names, so it can be tested on people it has never seen. Writers were split before pairs were built, so no writer or image appears in more than one of training, validation and test. If pairs had been made first and then split at random, the same writers (and even the same images) would appear on both sides and the test would no longer measure matching of new people.

Leakage and optimism checks.

- Pixels were divided by the fixed value 255, so no scaling parameters were learned from validation or test data.
- The CNN's hyperparameters and the Siamese threshold were chosen on validation data, and each test set was evaluated once.
- Using one session for validation and one for test means each estimate rests on a single new occasion. Per-writer accuracy changes a lot between the two: writer 7 scores 30% in session 4 and 100% in session 5, and writer 6 scores 24% and 79%. Overall accuracy is more stable (55.1% versus 56.1%).
- The Siamese estimate rests on only four test writers, which give six different-writer combinations. Matching difficulty depends mainly on *which* two writers are compared. Among the validation writers, pairs from writers 3 and 18 are wrongly matched only 0.6% of the time, while pairs from writers 3 and 7 are wrongly matched 78% of the time. A different draw of test writers could move the result by many points, which explains the gap between validation and test. Rotating which writers form the test group would give a more stable estimate.
- The 8,000 test pairs are not 8,000 independent observations: images are reused across pairs and come from only 20 recordings (4 writers by 5 sessions).

5. Limited hyperparameter optimisation (CNN)

Following the deck-3 procedure, we trained the CNN for each combination of learning rate (0.001, 0.0001) and dropout rate (0.2, 0.5), with the same seed, architecture, data and 20 epochs, and chose the combination with the highest validation accuracy. To see how much of any difference is just training randomness, we then retrained the chosen configuration with three different seeds.

Table 5: CNN grid search (validation = session 4). Configuration 4 has the highest validation accuracy.

Config	Learning rate	Dropout	Training acc.	Same-session check	Validation acc.	Validation loss
1	0.0010	0.2	100.0%	100.0%	51.5%	4.47
2	0.0001	0.2	100.0%	100.0%	54.5%	2.88
3	0.0010	0.5	100.0%	100.0%	52.6%	4.64
4	0.0001	0.5	100.0%	100.0%	55.1%	2.47

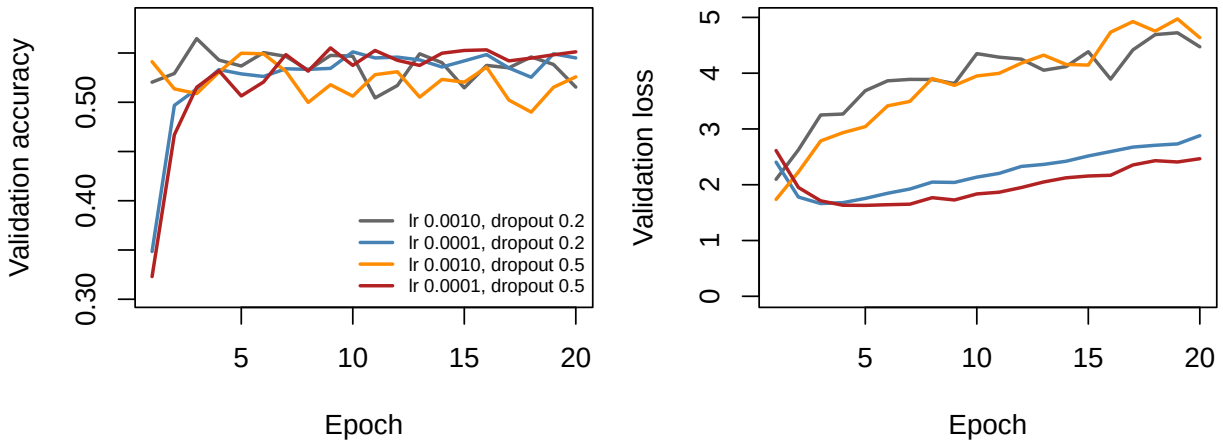


Figure 8: Validation accuracy (left) and validation loss (right) by epoch for the four configurations.

Table 6: Chosen configuration retrained with different seeds.

Seed	Validation accuracy
123	55.1%
456	53.1%
789	53.6%

Sensitivity.

- **Validation accuracy is not very sensitive.** It ranges from 51.5% to 55.1%, a spread of 3.6 percentage points. The smaller learning rate is better at both dropout rates (by 3.0 and 2.5 points). Raising dropout from 0.2 to 0.5 changes validation accuracy by 1.0 and 0.6 points. Changing only the seed moves the chosen configuration's validation accuracy by 2.0 points, 56% of the grid's spread. Only the learning-rate effect is consistent and larger than the seed-to-seed variation; the dropout effect cannot be told apart from training noise.
- **Validation loss is very sensitive.** At learning rate 0.001 it averages 4.55, against 2.67 at 0.0001. With the larger learning rate, validation loss is lowest after epoch 1 and then climbs, while validation accuracy barely changes (Figure 8). The model becomes increasingly *confident* about the wrong writer on the new session. The slower learning rate delays this overfitting but does not remove it.

- **Training and same-session accuracy are hardly sensitive at all.** Every configuration scores 100% (after rounding) on the same-session check. No setting in this grid closes the gap between seen and new sessions, because that gap is not a tuning problem: the training data contain only three recording conditions per writer.

The chosen configuration (learning rate 0.0001, dropout 0.5) scored 56.1% on the test session. Given the seed check, we would expect the other configurations to perform within a few points of it.

6. Comparison and discussion

What each model answers. The CNN answers "which of these 20 known people wrote this digit?" (chance 5%); the Siamese network answers "were these two digits written by the same person?" for people it has never seen (chance 50%). Their accuracies are not on the same scale and should not be ranked against each other. They are complementary: the CNN needs examples of every writer in advance but names them; the Siamese network needs no enrolment but only compares.

Table 7: Writer information by digit. CNN: test-session identification accuracy. Siamese: extra test sets of 2,000 same-digit pairs per digit from the unseen writers, at the validation-chosen threshold.

Digit	CNN	Same pairs accepted	Different pairs rejected	Siamese balanced acc.	Siamese AUC
0	60.0%	91.3%	59.5%	75.4%	0.86
1	51.9%	78.3%	49.2%	63.8%	0.72
2	48.6%	80.2%	70.4%	75.3%	0.85
3	66.5%	94.3%	55.2%	74.8%	0.88
4	53.6%	96.8%	29.9%	63.3%	0.81

Which digits carry more writer information? Ranked from most to least informative, the CNN orders the digits 3, 0, 4, 1, 2 and the Siamese network orders them 0, 2, 3, 1, 4. Both models place digits 0 and 3 among their three best and digit 1 among their two weakest. This fits the images: a one is mostly a single vertical stroke with few places for style to show, whereas digits with curves give width, roundness and closure room to vary between writers. The models disagree most about digit 2. The Siamese network's weakest digit is 4: it accepts 97% of same-writer pairs but rejects only 30% of different-writer pairs, so the four test writers' 4s sit close together in the embedding. The threshold-free AUC ranks the digits 3, 0, 2, 4, 1, with digit 1 lowest (0.72). 4 of the five single-digit AUCs are above the AUC of 0.76 for the main test set, whose pairs mostly show two different digits. A digit's ranking therefore depends on the model and on which writers are being compared, and only the broad pattern should be taken seriously.

Comparing whole pairs makes the same point. On the main test set, same-writer pairs are accepted 87.8% of the time when both images show the same digit and 80.2% when they show different digits. A shared character makes style easier to compare, but the network still matches across digits well above chance, so some of what it learns is a general style (size, slant, stroke width) rather than the shape of one digit.

Table 8: Effect of recording session on performance.

Comparison	Images / pairs	Correct
CNN: unseen images from training sessions (1-3)	3000	100.0%
CNN: new session 4 (validation)	5000	55.1%
CNN: new session 5 (test)	5000	56.1%
Siamese: same-writer pairs from the same session	804	91.4%
Siamese: same-writer pairs from different sessions	3240	79.3%

How session changes affect performance. The session effect is the main finding of this assignment (Table 8). The CNN identifies writers 100% of the time in recordings it has seen, even from images it has never trained on, but only 56% of the time in a new session. That gap of 44 percentage points comes from learning the particular look of each recording rather than the person. The Siamese network shows the same pattern on unseen writers: a true match is accepted 91.4% of the time when both images come from one session but 79.3% when they come from different sessions.

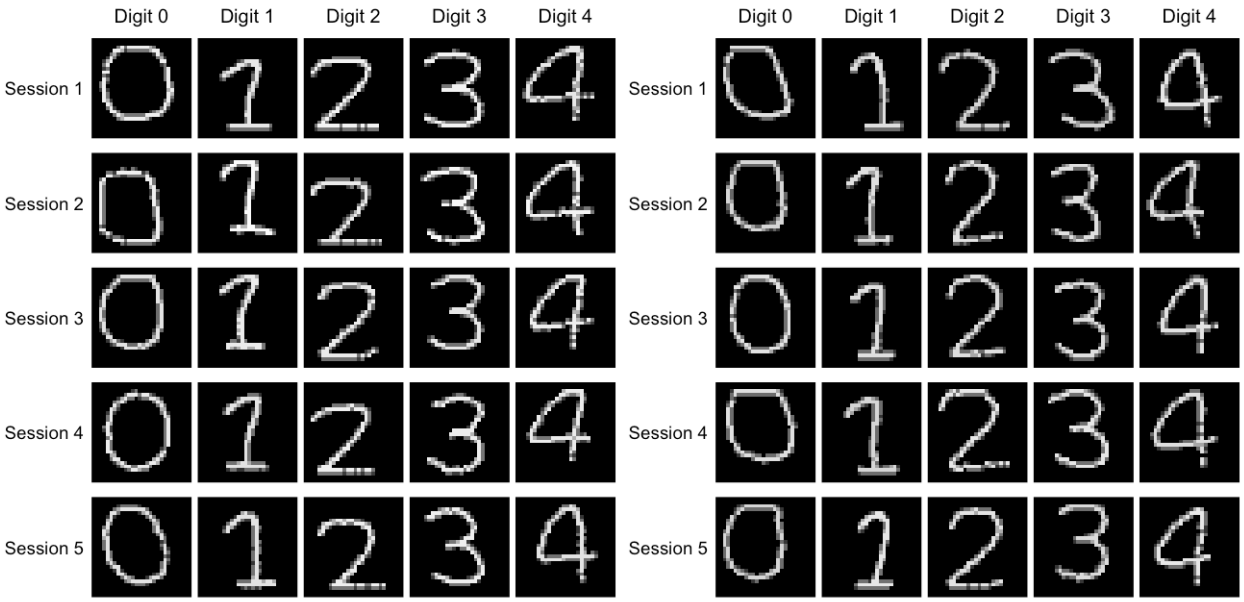


Figure 9: Writer 5 (left) and writer 13 (right) across their five sessions. The CNN labelled 198 of writer 5's 250 session-5 images as writer 13.

The session change also explains the CNN's worst errors. The model judged writer 5's session-5 digits to be closer to writer 13 than to writer 5's own training sessions (Figure 9): 198 of writer 5's 250 test images were labelled as writer 13, and 95 of writer 13's images were labelled as writer 5. Writer 10's session-5 writing was taken for writer 2 in 222 of 250 images. Which writers suffer depends on the session: writer 7 scores 30% in session 4 but 100% in session 5. When a writer's style drifts towards someone else's, the classifier assigns much of that session to the other person.

What would help. The notes suggest more varied training data. Here that means more sessions per writer, since three sessions are too few to learn what stays constant about a person, and possibly

data augmentation (rotation, cropping) to imitate small changes in size and slant. Larger networks or longer training would not help: the training curves show the model already fits the training sessions almost perfectly.

Conclusion. The CNN identifies the writer of a single digit from a new session 56% of the time (chance 5%). The Siamese network matches pairs from unseen writers with 68% balanced accuracy (chance 50%). Digits 0 and 3 carry the most writer information for both models and digit 1 the least, although the exact ranking depends on the model. Changes between recording sessions are the main obstacle in this data: performance on familiar sessions is far higher than on new ones. Only splits that hold out whole sessions or writers reveal this.

Plagiarism declaration

We know that plagiarism is wrong. Plagiarism is to use another's work and pretend that it is one's own. We have used the required convention for citation and referencing: each contribution to, and quotation in, this assignment from the work of other people has been attributed and has been cited and referenced. This assignment is our own work. We have not allowed, and will not allow, anyone to copy our work with the intention of passing it off as their own work.

Dakalo Manenzhe (MNNDAK002) and Troy Bisnath (BSNTRO001)

AI use statement

We used generative AI tools in preparing this assignment, as follows.

- **[Tool name], used by Dakalo:** [what it was used for].
- **Claude (Anthropic), used by Troy:** suggesting additional analyses, and help with the Git/GitHub workflow.

All AI output was checked and edited by us. The analysis choices, interpretation of results and final text are our own, and we take full responsibility for the content of this report.

Appendix A: code

All results in this report come from the code below, which runs when the document is rendered. Methods follow the course notes (the CNN notes, slide decks 1–3 and the metric-learning notes).

```
# Task 1: load the data, check it and draw example images
handwriting = readRDS("data/handwriting.rds")
metadata = handwriting$metadata
images = handwriting$images   # 25000 x 28 x 28 x 1, pixel values 0-255

stopifnot(
  dim(images)[1] == nrow(metadata),
  all(metadata$image_id == seq_len(nrow(metadata))),
  all(with(metadata, table(person_id, session, digit)) == 50),
  !anyNA(images), !anyNA(metadata)
```

```
)

find_image = function(writer, session, digit, replicate = 1) {
  which(metadata$person_id == writer & metadata$session == session &
          metadata$digit == digit & metadata$replicate == replicate)
}

# Matrix of image numbers, one row per value of `rows` and one column per value of `cols`
image_index = function(rows, cols, pick) {
  t(sapply(rows, function(r) sapply(cols, function(cl) pick(r, cl))))
}

draw_digit = function(i) {
  pixels = images[i, , , 1]
  image(t(pixels[28:1, ]), col = gray(seq(0, 1, length.out = 256)),
        zlim = c(0, 255), axes = FALSE, asp = 1, useRaster = TRUE)
}

image_grid = function(file, index, row_labels, col_labels, width, height) {
  png(file, width = width, height = height, res = 150)
  par(mfrow = dim(index), mar = rep(0.15, 4), oma = c(0.5, 4.5, 2.2, 0.5))
  for (r in seq_len(nrow(index))) {
    for (cl in seq_len(ncol(index))) {
      draw_digit(index[r, cl])
      if (cl == 1) mtext(row_labels[r], side = 2, line = 0.4, las = 1, cex = 0.75)
      if (r == 1) mtext(col_labels[cl], side = 3, line = 0.4, cex = 0.8)
    }
  }
  dev.off()
}

# One writer's five sessions, replicate 1 of each digit
sessions_figure = function(writer) {
  file = sprintf("images/sessions_writer_%02d.png", writer)
  index = image_index(1:5, 0:4, function(s, d) find_image(writer, s, d))
  image_grid(file, index, paste("Session", 1:5), paste("Digit", 0:4), 650, 650)
  file
}

# All writers and digits from session 1, split into two images of ten writers
index = image_index(1:20, 0:4, function(w, d) find_image(w, 1, d))
for (first in c(1, 11)) {
  writers = first:(first + 9)
  image_grid(sprintf("images/writers_%02d_%02d.png", first, first + 9), index[writers, ],
             paste("Writer", writers), paste("Digit", 0:4), 650, 1250)
}
```

```r
sessions_figure(1)

index = image_index(0:4, 1:8, function(d, r) find_image(1, 1, d, r))
image_grid("images/within_session.png", index, paste("Digit", 0:4),
           paste("Rep", 1:8), 1000, 650)

# Simple size measures: ink is the mean pixel value; height and width count the
# rows and columns that contain a pixel brighter than 50
pixels = images[, , , 1]
inked = pixels > 50
summ = data.frame(
  metadata,
  ink = apply(pixels, 1, mean),
  height = apply(inked, 1, function(a) sum(rowSums(a) > 0)),
  width = apply(inked, 1, function(a) sum(colSums(a) > 0))
)
# Tasks 2 and 5: CNN writer identification and the hyperparameter grid
library(keras3)
use_backend("tensorflow")

epochs = 20
batch_size = 64
x = images / 255
y = metadata$person_id - 1   # Keras needs classes numbered from 0

# Sessions 1-3 are for training, session 4 for validation and session 5 for testing.
# Replicates 41-50 of sessions 1-3 are held back to check accuracy on sessions the
# model has already seen.
rows = list(
  train = which(metadata$session %in% 1:3 & metadata$replicate <= 40),
  check = which(metadata$session %in% 1:3 & metadata$replicate > 40),
  val   = which(metadata$session == 4),
  test  = which(metadata$session == 5)
)
x_split = lapply(rows, function(r) x[r, , , , drop = FALSE])
y_split = lapply(rows, function(r) to_categorical(y[r], 20))

write.csv(data.frame(split = c("training", "same-session check", "validation", "test"),
                     images = lengths(rows)),
          "results/cnn_split_sizes.csv", row.names = FALSE)

build_cnn = function(learning_rate, dropout_rate) {
  input = layer_input(shape = c(28, 28, 1))
  output = input |>
    layer_conv_2d(filters = 32, kernel_size = c(3, 3), activation = "relu") |>
    layer_max_pooling_2d(pool_size = c(2, 2)) |>
    layer_conv_2d(filters = 64, kernel_size = c(3, 3), activation = "relu") |>
```

```
    layer_max_pooling_2d(pool_size = c(2, 2)) |>
    layer_flatten() |>
    layer_dense(units = 128, activation = "relu") |>
    layer_dropout(rate = dropout_rate) |>
    layer_dense(units = 20, activation = "softmax")
  model = keras_model(inputs = input, outputs = output)
  model |> compile(
    loss = "categorical_crossentropy",
    optimizer = optimizer_adam(learning_rate = learning_rate),
    metrics = c("accuracy")
  )
  model
}

predict_writer = function(model, split) {
  max.col(predict(model, x_split[[split]], verbose = 0)) - 1
}
accuracy = function(model, split) mean(predict_writer(model, split) == y[rows[[split]]])

# Grid search: every configuration starts from the same seed and is compared on
# validation accuracy
grid = expand.grid(learning_rate = c(0.001, 0.0001), dropout_rate = c(0.2, 0.5))
grid$config = seq_len(nrow(grid))
grid[c("train_accuracy", "check_accuracy", "val_accuracy", "val_loss",
       "lowest_val_loss_epoch")] = NA
histories = list()

for (i in grid$config) {
  set_random_seed(123)
  model = build_cnn(grid$learning_rate[i], grid$dropout_rate[i])
  history = model |> fit(
    x_split$train, y_split$train,
    epochs = epochs, batch_size = batch_size, shuffle = TRUE,
    validation_data = list(x_split$val, y_split$val), verbose = 0
  )
  histories[[i]] = history$metrics
  grid$train_accuracy[i] = accuracy(model, "train")
  grid$check_accuracy[i] = accuracy(model, "check")
  grid$val_accuracy[i] = accuracy(model, "val")
  grid$val_loss[i] = tail(history$metrics$val_loss, 1)
  grid$lowest_val_loss_epoch[i] = which.min(history$metrics$val_loss)
  save_model(model, sprintf("models/cnn_config_%d.keras", i), overwrite = TRUE)
}
write.csv(grid, "results/cnn_grid.csv", row.names = FALSE)
saveRDS(histories, "results/cnn_histories.rds")
best = grid$config[which.max(grid$val_accuracy)]
```

```
# Retrain the chosen configuration with other seeds to see how much
# validation accuracy changes from run to run
seeds = data.frame(seed = c(123, 456, 789), val_accuracy = NA)
for (i in seq_len(nrow(seeds))) {
  set_random_seed(seeds$seed[i])
  model = build_cnn(grid$learning_rate[best], grid$dropout_rate[best])
  model |> fit(x_split$train, y_split$train, epochs = epochs,
               batch_size = batch_size, shuffle = TRUE, verbose = 0)
  seeds$val_accuracy[i] = accuracy(model, "val")
}
write.csv(seeds, "results/cnn_seed_check.csv", row.names = FALSE)

# The test session is used once, for the chosen model only
model = load_model(sprintf("models/cnn_config_%d.keras", best))
test_eval = model |> evaluate(x_split$test, y_split$test, verbose = 0)
write.csv(data.frame(selected_config = best, test_loss = test_eval$loss,
                     test_accuracy = test_eval$accuracy),
          "results/cnn_test_evaluate.csv", row.names = FALSE)

split_names = c(check = "same-session check", val = "validation", test = "test")
cnn_pred = do.call(rbind, lapply(names(split_names), function(s) {
  r = rows[[s]]
  data.frame(image_id = metadata$image_id[r], split = split_names[[s]],
             actual_writer = metadata$person_id[r],
             predicted_writer = predict_writer(model, s) + 1,
             metadata[r, c("digit", "session", "replicate")])
}))
cnn_pred$correct = cnn_pred$actual_writer == cnn_pred$predicted_writer
write.csv(cnn_pred, "results/cnn_predictions.csv", row.names = FALSE)
# Task 3: Siamese network with contrastive loss, following the metric-learning notes
labels = metadata$person_id

# Split the writers first, so no writer appears in more than one set of pairs
set.seed(2026)
writers = sample(1:20)
writer_sets = list(train = sort(writers[1:12]), val = sort(writers[13:16]),
                   test = sort(writers[17:20]))
ids = lapply(writer_sets, function(w) which(labels %in% w))

# make_pairs() and make_batch() from the notes: the second image comes from the
# same writer about half the time
make_pairs = function(ids, n, class_labels = labels) {
  a = sample(ids, n, replace = TRUE)
  b = vapply(a, function(i) {
    same = runif(1) < .5
    pool = ids[class_labels[ids] == class_labels[i]]
    if (!same) pool = ids[class_labels[ids] != class_labels[i]]
```

```
    sample(setdiff(pool, i), 1)
  }, integer(1))
  cbind(a, b)
}
make_batch = function(p, images = x, class_labels = labels) {
  y = matrix(as.integer(class_labels[p[, 1]] == class_labels[p[, 2]]), ncol = 1)
  list(left = images[p[, 1], , , , drop = FALSE],
       right = images[p[, 2], , , , drop = FALSE], y = y)
}

set.seed(25)
pairs = list(train = make_pairs(ids$train, 20000), val = make_pairs(ids$val, 4000),
             test = make_pairs(ids$test, 8000))
batches = lapply(pairs, make_batch)

# Shared encoder: each image becomes a 32-number embedding
image = layer_input(shape = c(28, 28, 1))
embedding = image |>
  layer_conv_2d(32, 3, activation = "relu") |>
  layer_max_pooling_2d(2) |>
  layer_conv_2d(64, 3, activation = "relu") |>
  layer_global_average_pooling_2d() |>
  layer_dense(32)
set_random_seed(123)
encoder = keras_model(image, embedding)

# Both images go through the same encoder; the output is the distance between them
a = layer_input(shape = c(28, 28, 1))
b = layer_input(shape = c(28, 28, 1))
distance = list(encoder(a), encoder(b)) |>
  layer_lambda(f = function(z) {
    op_sqrt(op_sum(op_square(z[[1]] - z[[2]]), axis = 2, keepdims = TRUE) + 1e-7)
  })
siamese = keras_model(list(a, b), distance)

contrastive_loss = function(y, d) {
  op_mean(y * op_square(d) + (1 - y) * op_square(op_maximum(1 - d, 0)))
}
siamese |> compile(optimizer = optimizer_adam(1e-3), loss = contrastive_loss)
siamese_history = siamese |> fit(
  list(batches$train$left, batches$train$right), batches$train$y,
  validation_data = list(list(batches$val$left, batches$val$right), batches$val$y),
  epochs = 20, batch_size = batch_size, verbose = 0
)
saveRDS(siamese_history$metrics, "results/siamese_history.rds")
save_model_weights(siamese, "models/siamese.weights.h5", overwrite = TRUE)
```

```
pair_distance = function(batch) {
  as.numeric(predict(siamese, list(batch$left, batch$right), verbose = 0))
}

# Choose the threshold that gives the best balanced accuracy on the validation
# pairs; a pair is called "same writer" when its distance is below it
valid_distance = pair_distance(batches$val)
valid_same = as.logical(batches$val$y[, 1])
balanced_accuracy_at = function(cut) {
  predicts_same = valid_distance < cut
  (mean(predicts_same[valid_same]) + mean(!predicts_same[!valid_same])) / 2
}
candidates = sort(unique(valid_distance))
scores = vapply(candidates, balanced_accuracy_at, numeric(1))
threshold = candidates[which.max(scores)]

pair_table = function(p, batch, d, set) {
  data.frame(set = set, left = p[, 1], right = p[, 2],
             same_writer = batch$y[, 1], distance = d,
             predicted_same = as.integer(d < threshold),
             writer_left = labels[p[, 1]], writer_right = labels[p[, 2]],
             digit_left = metadata$digit[p[, 1]], digit_right = metadata$digit[p[, 2]],
             session_left = metadata$session[p[, 1]],
             session_right = metadata$session[p[, 2]])
}
siamese_pairs = rbind(
  pair_table(pairs$val, batches$val, valid_distance, "validation"),
  pair_table(pairs$test, batches$test, pair_distance(batches$test), "test")
)

# Extra test sets with one digit each, to compare digits
for (d in 0:4) {
  p = make_pairs(ids$test[metadata$digit[ids$test] == d], 2000)
  batch = make_batch(p)
  siamese_pairs = rbind(siamese_pairs,
                        pair_table(p, batch, pair_distance(batch), paste("digit", d)))
}
write.csv(siamese_pairs, "results/siamese_pairs.csv", row.names = FALSE)
saveRDS(list(train_writers = writer_sets$train, val_writers = writer_sets$val,
             test_writers = writer_sets$test, threshold = threshold,
             val_balanced_accuracy = max(scores),
             n_train_pairs = nrow(pairs$train),
             train_same_share = mean(batches$train$y)),
        "results/siamese_design.rds")

# Record the software used
tf = reticulate::import("tensorflow")
```

```r
writeLines(c(R.version.string,
             paste("keras3", as.character(packageVersion("keras3"))),
             paste("TensorFlow", tf$`__version__`),
             paste("Keras", tf$keras$`__version__`)),
           "results/versions.txt")
writeLines(capture.output(sessionInfo()), "results/session_info.txt")
```

Appendix B: software

Models were trained with R version 4.3.3 (2024-02-29), keras3 1.2.0, TensorFlow 2.21.0 and Keras 3.15.1 (Python backend). The full R session information is saved in `results/session_info.txt`.